

FL-02 Prompt Iteration Log

FlyRank AI Internship

Assignment FL-02 - Prompting Fundamentals on Real Tasks v2

Name: Achyuta Biswal **Track:** Backend AI Engineering **Related Assignment:** FL-01 - Employee Task Management System

1. Introduction

What Prompt Engineering Is

Prompt engineering is the practice of deliberately designing the instructions given to an AI model so that its output matches a specific intent, audience, and quality bar. It is not about “asking nicely” — it is about controlling the variables that determine output quality: the goal being communicated, the context the model has access to, the format the response should take, the boundaries it must respect, and the standard it should be judged against. Two prompts describing the same underlying task can produce wildly different results depending on how much of that information is made explicit versus left for the model to guess.

Why Iterative Prompting Improves AI Output

A single prompt, no matter how carefully worded, rarely captures every requirement on the first attempt. Iterative prompting treats prompt-writing as a feedback loop rather than a one-shot instruction: each version is evaluated against the previous one, a specific weakness is identified, and a single targeted improvement is applied. This isolates cause and effect — instead of guessing which change helped, the engineer can point to exactly which addition (a role, a piece of context, a format constraint) produced which improvement. Over several iterations, this produces a prompt that is deliberately engineered rather than accidentally adequate.

Purpose of This Assignment

This assignment applies iterative prompting to a real, personally relevant task carried over from FL-01: explaining the JWT Role-Based Authentication implementation of an Employee Task Management System, for the purpose of technical interview preparation. The goal is to document six successive prompt versions, each applying one specific prompting technique, and to honestly evaluate what each technique changed about the output. The assignment also compares how two different AI models (Claude and ChatGPT) respond to the same final prompt, and concludes with a reusable, project-agnostic prompt template and a personal reflection on what the exercise revealed about prompt engineering as a skill.

2. Real Task

Project Description

The real task is drawn directly from my FL-01 assignment: the **Employee Task Management System**, a backend application that allows organizations to assign, track, and manage employee tasks with role-differentiated access (e.g., Admin, Manager, Employee). Authentication and authorization are handled using stateless JWT-based security integrated with Spring Security.

Technology Stack

Component	Technology
Language	Java 23
Framework	Spring Boot
Security	Spring Security
Authentication	JWT (JSON Web Tokens)
Data Access	JDBC
Database	MySQL

Goal of This Prompting Exercise

The purpose of prompting an AI model on this task is **not** to generate new code, but to produce a clear, technically accurate, interview-ready explanation of how JWT Role-Based Authentication works in this specific system — covering token generation, token validation, role extraction, and how Spring Security enforces role-based access control on protected endpoints. The target audience is an interviewer assessing backend engineering depth, so the ideal output must balance technical precision with clear verbal explainability.

3. Prompt Iteration Log

Version 1 — Naive Prompt

Technique: Naive Prompt

Explain JWT authentication in Spring Boot.

Example Output

JWT (JSON Web Token) is a compact, URL-safe token format used for authentication. In Spring Boot, a user logs in with a username and password, and the server generates a JWT signed with a secret key. This token is then sent back to the client, which includes it in the Authorization header of future requests. Spring Security uses a filter to validate the token on each request and allows or denies access based on whether the token is valid.

Observation

This response is generic and could apply to almost any Spring Boot project using JWT — it says nothing about **role-based** authorization, nothing about the specific stack (JDBC, MySQL, Java 23), and nothing about the Employee Task Management System itself. It reads like a textbook definition rather than an explanation of *my* implementation, which makes it unusable for interview preparation since it wouldn't survive a single follow-up question about the actual project. The core weakness is that the prompt gave the model no identity, no context, and no goal beyond “explain,” so it defaulted to the most generic possible answer.

Version 2 — Role Assignment

Technique: Role Assignment

You are a senior backend engineer mentoring a junior developer.
Explain JWT authentication in Spring Boot.

Example Output

Great question. Let's break down JWT authentication the way I'd explain it to someone new on my team.

When a user logs in, the backend verifies their credentials and, if valid, issues a JWT — think of it as a signed, tamper-proof ID card. That token contains claims (like the username and, often, roles) and is signed with a secret key so the server can later verify it hasn't been altered.

On every subsequent request, the client sends this token in the Authorization header. A filter intercepts the request before it reaches your controllers, validates the token's signature and expiration, and if everything checks out, sets the authenticated user in the security context so Spring Security knows who's making the request.

The nice part about this approach is that the server doesn't need to store session state — it's fully stateless, which scales better across multiple server instances.

Observation

Assigning the model a role (senior engineer mentoring a junior developer) noticeably changed the **tone and framing** of the response — it became more explanatory and pedagogical, using analogies (“signed, tamper-proof ID card”) and explicitly noting the *why* behind design decisions (statelessness, scalability) rather than just listing mechanics. This is closer to how a candidate should actually speak in an interview. However, the content is still generic to JWT-in-Spring-Boot as a concept — it still has no awareness of the Employee Task Management System, role-based access control specifically, or the JDBC/MySQL persistence layer, so it remains disconnected from the real project.

Version 3 — Context and Motivation

Technique: Context and Motivation

You are a senior backend engineer mentoring a junior developer.

I built an Employee Task Management System using Java 23, Spring Boot, Spring Security, JWT, JDBC, and MySQL. The system has three roles – ADMIN, MANAGER, and EMPLOYEE – and different endpoints are restricted based on these roles.

I am preparing for a technical interview and need to explain how JWT Role-Based Authentication works in this specific project, including how roles are embedded in the token and enforced on protected endpoints.

Example Output

Let's walk through how this works in your system specifically.

When a user logs in, your `AuthController` passes the credentials to Spring Security's `AuthenticationManager`, which verifies them against the user record pulled via JDBC from MySQL. Once authenticated, your `JwtService` generates a token – and critically, this is where role-based access starts: the user's role (ADMIN, MANAGER, or EMPLOYEE) is embedded directly into the token's claims, typically under a "roles" or "authorities" claim.

On each subsequent request, your `JwtAuthenticationFilter` extracts the token from the Authorization header, validates its signature and expiration, and then parses the role claim to reconstruct a list of `GrantedAuthority` objects. These are attached to the Authentication object in the `SecurityContext`.

Your `SecurityFilterChain` configuration is what actually enforces role restrictions – for example, using `.requestMatchers("/api/admin/**").hasRole("ADMIN")` – so even if a valid token is presented, Spring Security will block access if the embedded role doesn't match what the endpoint requires.

This design means role enforcement happens at two layers: token issuance (what role gets embedded) and request-time authorization (what role is required) – both must agree for access to be granted.

Observation

Adding real project context (the specific roles, the specific stack, and the explicit mention of role-based enforcement) is the single biggest jump in relevance so far — the response now correctly ties the explanation to **this** system's actual components (`AuthController`, `JwtService`, `JwtAuthenticationFilter`, `SecurityFilterChain`) instead of speaking abstractly about "a" Spring Boot app. It also correctly separated the two enforcement layers (token issuance vs. request-time authorization), which is exactly the kind of nuance an interviewer would probe for. The remaining gap is consistency and predictability — since no format or example was given, the structure of this explanation (what it emphasizes, what class names it invents) could vary significantly between runs, which is a problem for someone trying to rehearse a repeatable interview answer.

Version 4 — Few-Shot Prompting

Technique: Few-Shot Prompting

You are a senior backend engineer mentoring a junior developer.

I built an Employee Task Management System using Java 23, Spring Boot, Spring Security, JWT, JDBC, and MySQL, with three roles: ADMIN, MANAGER, and EMPLOYEE.

When explaining a backend security concept, follow this style:

Example 1:

Concept: Password hashing

Explanation: "Passwords are never stored in plain text. When a user registers, BCrypt hashes the password with a random salt before it's persisted via JDBC. During login, the entered password is hashed again and compared to the stored hash – this way, even a database breach doesn't expose usable credentials."

Example 2:

Concept: Stateless sessions

Explanation: "Because JWTs carry all necessary identity information within the token itself, the server doesn't need to maintain session state in memory or in the database. This lets the application scale horizontally without needing sticky sessions or a shared session store."

Now, using this same style, explain how JWT Role-Based Authentication is implemented in my project, including how roles are embedded and enforced.

Example Output

Concept: JWT Role-Based Authentication

Explanation: "When a user logs in, their credentials are verified against the MySQL-backed user table via JDBC, and their assigned role – ADMIN, MANAGER, or EMPLOYEE – is fetched alongside their identity. The JwtService embeds this role directly into the token as a claim before signing it, so the role travels with the token rather than requiring a database lookup on every request. On incoming requests, the JwtAuthenticationFilter extracts and validates the token, then reconstructs the user's authorities from the embedded role claim. Spring Security's SecurityFilterChain then compares this authority against the role required by the requested endpoint – for instance, an EMPLOYEE-role token attempting to hit an ADMIN-only endpoint is rejected with a 403, even though the token itself is perfectly valid. This keeps authorization decisions fast and stateless, since the role never needs to be re-fetched from the database after login."

Observation

Providing two worked examples of the desired explanation style visibly improved **consistency of structure** — the output now follows the same “Concept / Explanation” pattern and the same density of technical detail as the examples, rather than free-form prose that might be longer or shorter depending on the run. This matters directly for interview prep: a candidate wants a repeatable, memorizable explanation, not a novel essay every time they ask. The output also picked up the stylistic habit from Example 2 of ending with a “why this matters” sentence (statelessness), showing the model generalized the pattern rather than just the topic. The main limitation is that this format is a single dense paragraph — for actual studying, a step-by-step or sectioned breakdown would be easier to review than one continuous block of text.

Version 5 — Output Structure

Technique: Output Structure

You are a senior backend engineer mentoring a junior developer.

I built an Employee Task Management System using Java 23, Spring Boot, Spring Security, JWT, JDBC, and MySQL, with three roles: ADMIN, MANAGER, and EMPLOYEE.

Explain how JWT Role-Based Authentication is implemented in this project, structured as follows:

Overview
(2-3 sentence summary)

Step-by-Step Flow
(numbered list of what happens from login to authorization)

Key Components
(table with Component | Responsibility)

Common Interview Follow-Up Questions
(3 likely follow-up questions with one-line answers)

Example Output

Overview

JWT Role-Based Authentication in this system allows the server to verify who a user is and what they're allowed to do, without maintaining session state, by embedding the user's role directly into a signed token issued at login.

Step-by-Step Flow

1. User submits credentials to the login endpoint.
2. Spring Security's AuthenticationManager verifies credentials against the MySQL user table via JDBC.
3. On success, JwtService generates a signed JWT containing the username and role (ADMIN, MANAGER, or EMPLOYEE) as claims.
4. The client stores this token and sends it in the Authorization header on subsequent requests.
5. JwtAuthenticationFilter intercepts each request, validates the token's signature and expiration, and extracts the role claim.
6. The extracted role is converted into a GrantedAuthority and attached to the SecurityContext.
7. SecurityFilterChain rules (e.g., hasRole("ADMIN")) compare the authenticated user's role against what the endpoint requires.
8. Access is granted or a 403 Forbidden is returned.

Key Components

Component	Responsibility
AuthController	Accepts login requests, delegates authentication
JwtService	Generates and signs tokens, embeds role claims
JwtAuthenticationFilter	Validates tokens on each request, extracts roles
SecurityFilterChain	Enforces role requirements per endpoint
UserRepository (JDBC)	Fetches user credentials and role from MySQL

Common Interview Follow-Up Questions

1. **What happens if the token is expired?** The filter rejects it before reaching the controller, returning a 401 Unauthorized.
2. **Where is the role stored – in the token or the database?** In the token as a claim, so it doesn't require a database lookup per request.
3. **How would you revoke a compromised token before it expires?** Typically via a token blacklist or short expiration with refresh tokens.

Observation

Specifying an explicit output structure dramatically improved **readability and study-usability** — the response is now scannable in seconds, with a clear entry point (Overview), a mechanism to follow (Step-by-Step Flow), a reference table (Key Components), and even anticipates the interview format itself (Follow-Up Questions). This is a qualitatively different kind of output than Version 4: it's no longer just an explanation, it's a study artifact. The Key Components table in particular gives a fast mental map of class responsibilities that would be tedious to extract from a paragraph. One shortcoming is that the "Common Interview Follow-Up Questions" section, while useful, was invented by the model without being asked to justify *why* those three questions specifically were chosen — it would help to know they reflect genuinely common interview probes rather than arbitrary examples.

Version 6 — Step Decomposition

Technique: Step Decomposition

You are a senior backend engineer mentoring a junior developer preparing for a technical interview.

I built an Employee Task Management System using Java 23, Spring Boot, Spring Security, JWT, JDBC, and MySQL, with three roles: ADMIN, MANAGER, and EMPLOYEE.

Break this explanation into the following sequential steps, and complete each step fully before moving to the next:

Step 1: Explain how credentials are authenticated at login.
Step 2: Explain how the JWT is generated and what claims it contains.
Step 3: Explain how the token is validated on subsequent requests.
Step 4: Explain how the role claim is converted into a Spring Security authority.
Step 5: Explain how endpoint-level role restrictions are enforced.
Step 6: Summarize the full flow in 3 sentences suitable for a spoken interview answer.

Example Output

Step 1 – Authenticating credentials at login
When a login request hits AuthController, the raw username and password are passed to Spring Security's AuthenticationManager. Internally, this delegates to a DaoAuthenticationProvider, which loads the user via a JDBC query against the MySQL users table and compares the submitted password against the stored BCrypt hash. If they match, authentication succeeds.

Step 2 – Generating the JWT and its claims
Once authenticated, JwtService builds a token containing the subject (username), an issued-at timestamp, an expiration timestamp, and a "role" claim populated from the user's role in the database (ADMIN, MANAGER, or EMPLOYEE). The token is signed using a secret key (e.g., HMAC-SHA256), so any tampering with the payload invalidates the signature.

Step 3 – Validating the token on subsequent requests
Every request to a protected endpoint passes through JwtAuthenticationFilter before reaching the controller. This filter reads the Authorization header, extracts the token, verifies the signature against the secret key, and checks that the token hasn't expired. If either check fails, the request is rejected immediately.

Step 4 – Converting the role claim into a Spring Security authority
Once the token is verified, the filter reads the "role" claim and wraps it into a SimpleGrantedAuthority (typically prefixed with "ROLE_", e.g., "ROLE_ADMIN"). This authority, along with the username, is packaged into an Authentication object and set into the SecurityContextHolder, making the user "known" to Spring Security for the duration of the request.

Step 5 – Enforcing endpoint-level role restrictions
In the SecurityFilterChain configuration, specific URL patterns are mapped to required roles – for example, .requestMatchers("/api/admin/**").hasRole("ADMIN") or .requestMatchers("/api/manager/**").hasAnyRole("ADMIN", "MANAGER"). Spring Security compares the authorities attached to the current request against these rules before allowing the request to proceed to the controller. A mismatch results in a 403 Forbidden, even with a perfectly valid token.

Step 6 – Spoken summary
"In my project, a user logs in and receives a JWT that has their role embedded directly in it, so the server doesn't need to check the database on every request. A filter validates that token on each call, extracts the role, and Spring Security uses that role to decide whether the specific endpoint being accessed is allowed for that user – all without maintaining server-side session state."

Observation

Decomposing the task into six explicit, sequential steps produced the **most complete and interview-ready output** of all six versions, because it forced the model to fully develop each stage of the authentication flow individually instead of compressing several stages into a shared paragraph or bullet, as happened in earlier versions. Step 4 in particular — converting the role claim into a SimpleGrantedAuthority with the "ROLE_" prefix convention — is a level of implementation detail that never appeared in Versions 1 through 5, because there was no explicit instruction forcing the model to isolate that specific transition. The inclusion of Step 6 (a compressed, spoken-style summary) is uniquely valuable for interview prep, since it produces a rehearsable answer distinct from the technical deep-dive above it. This confirms that step decomposition works best not as a

replacement for the earlier techniques, but as a **combination point** — it only reached this depth because it was still carrying the role, context, and structural expectations established in Versions 2, 3, and 5.

4. Cross-Model Comparison

The final prompt from Version 6 was run on both **Claude** and **ChatGPT** to compare how each model handles the same fully-engineered prompt.

Criteria	Claude	ChatGPT
Tone	Measured, mentor-like, avoids over-selling the explanation; reads like a colleague reviewing your work	Energetic and encouraging, frequently uses affirming phrases (“Great question!”, “You’ve got this!”)
Technical Accuracy	Consistently correct on Spring Security internals (e.g., correctly distinguishes <code>AuthenticationManager</code> from <code>DaoAuthenticationProvider</code>)	Generally accurate but occasionally conflates filter-chain ordering details or oversimplifies the <code>SecurityContextHolder</code> mechanism
Structure	Follows the requested step structure precisely and rarely adds unrequested sections	Follows the structure but frequently adds extra unrequested elements (e.g., a bonus “Tips” or “Pro Tip” section)
Readability	Dense but precise; assumes the reader can handle technical density	Slightly more verbose with more transitional/explanatory sentences, which aids first-time readers but adds length
Practical Examples	Uses realistic, minimal code identifiers (e.g., <code>hasRole(“ADMIN”)</code>) without inventing excessive surrounding code	Tends to include more inline code snippets, sometimes more than requested, which can dilute focus from the explanation itself
Interview Preparation	The spoken-summary step is typically concise and closely matches natural spoken cadence	The spoken-summary step is sometimes noticeably longer than a natural spoken answer, requiring manual trimming
Weaknesses	Can be terser than necessary on the very first step if not explicitly told to elaborate	Prone to “over-helpfulness” — adding disclaimers, encouragement, or extra sections beyond what the structure specified

Detailed Comparison

The two models diverge most clearly in how strictly they respect an explicit structure once one is given. Claude tends to treat the numbered step list as a hard contract — it completes exactly what was asked in each step and stops, which makes its output easier to convert directly into interview flashcards without editing. ChatGPT, by contrast, tends to be generative beyond the requested structure: it will often append supplementary material (bonus tips, additional context, or reassurance) that was not requested, which can be useful for a first-time learner but becomes noise for someone who has already engineered the prompt precisely and wants a repeatable, trimmed answer.

On technical accuracy specifically for the Spring Security internals involved in this task, Claude showed a slightly stronger and more consistent grasp of the distinction between where authentication happens (`AuthenticationManager` / `DaoAuthenticationProvider`) versus where authorization is enforced (`SecurityFilterChain` matchers), whereas ChatGPT occasionally blurred this distinction by describing role-checking as happening “during” token validation rather than as a separate, later authorization step performed by the filter chain. This is a meaningful difference for interview preparation specifically, because interviewers frequently probe exactly this authentication-versus-authorization boundary, and an imprecise mental model here could cause a candidate to misspeak under pressure.

Where ChatGPT has a genuine edge is in accessibility for a first-time reader: its more verbose, encouraging tone and its habit of inserting extra illustrative snippets make it a better fit for someone using this material to *learn* the concept for the first time, rather than someone who already understands the system and is trying to *compress* it into a clean interview answer. Claude’s output, being terser and more literally obedient to the requested structure, is better suited to the specific goal of this assignment — producing a repeatable, rehearsable, technically precise answer — but would be a slightly worse starting point for a complete beginner to the concept.

Overall, neither model is unconditionally “better” — the right choice depends on whether the goal is initial learning (favor ChatGPT’s elaboration) or final interview-answer compression (favor Claude’s structural discipline).

5. Final Reusable Prompt Template

The following template generalizes the Version 6 (Step Decomposition) prompt into a reusable format that strips out all project-specific details from this assignment, so it can be applied to any real backend task.

```
You are a [ROLE] mentoring a junior developer preparing for a technical interview.

I built [PROJECT] using [TECH STACK].

Break this explanation into the following sequential steps, and complete each step fully before moving to the next:

Step 1: Explain [TASK 1].
Step 2: Explain [TASK 2].
Step 3: Explain [TASK 3].
Step 4: Explain [TASK 4].
Step 5: Explain [TASK 5].
Step 6: Summarize the full flow in 3 sentences suitable for a spoken interview answer.

Goal: [GOAL]
```

Placeholder Reference

Placeholder	Description	Example (from this assignment)
[ROLE]	The persona the AI should adopt	“senior backend engineer”
[PROJECT]	Name/description of the real project	“Employee Task Management System”
[TECH STACK]	Comma-separated technologies used	“Java 23, Spring Boot, Spring Security, JWT, JDBC, MySQL”
[TASK 1]–[TASK 5]	The individual stages of the concept being explained, in logical order	“how credentials are authenticated,” “how the JWT is generated,” etc.
[GOAL]	The end purpose of the explanation	“technical interview preparation”

This template is intentionally domain-agnostic — the same structure works equally well for explaining a caching layer, a message queue integration, a CI/CD pipeline, or a database migration strategy, simply by substituting the placeholders.

6. Reflection

Working through six iterations of the same underlying request made it clear that prompt quality is not a single lever but a stack of independent, additive decisions. The single most impactful technique in this exercise was **Step Decomposition** — it consistently produced the most complete and technically precise output because it prevented the model from compressing multiple distinct stages of the authentication flow (credential verification, token generation, token validation, authority conversion, and endpoint enforcement) into a single blended explanation. Every earlier version, no matter how much context or structure it had, still risked collapsing two or three of these stages together; forcing sequential, individually-completed steps eliminated that risk entirely.

At the same time, the exercise showed that step decomposition alone would not have worked well in Version 1 — it only produced its best result *because* it was layered on top of role assignment, real project context, and output structure established in earlier versions. This reframed how I think about prompting: it is less like picking the “best” single technique and more like sequencing techniques so each one resolves a specific ambiguity before the next is introduced. Iterative prompting changed the output from something generic and forgettable into something I could genuinely use to rehearse for a real interview — measurably more specific, more structurally consistent, and more aligned with how I would actually need to speak about this system under pressure.

This skill is valuable for software engineers because clear, structured communication about one’s own systems is itself an engineering deliverable — whether in interviews, design docs, or onboarding new teammates — and prompt engineering is, at its core, a disciplined practice in decomposing exactly what you need before asking for it, which is a transferable skill far beyond just working with AI models.

End of Assignment FL-02 Submission.